СОДЕРЖАНИЕ

ВВЕДЕНИЕ

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области и постановка задач

1.2 Моделирование и проектирование системы

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Разработка базы данных

2.2 Администрирование базы данных

2.3 Разработка программных модулей

2.4 Информационная безопасность

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А — Учётные записи для тестирования

ПРИЛОЖЕНИЕ Б — Листинг кода

ПРИЛОЖЕНИЕ В — Аннотация

ВВЕДЕНИЕ

Информационные системы управления фитнес-центрами представляют собой критически важный инструмент в современной индустрии здоровья и фитнеса. По мере роста конкуренции между фитнес-учреждениями и увеличения их сложности операционные процессы становятся все более сложными для управления вручную. Актуальность разработки специализированной информационной системы для фитнес-центра обусловлена множеством факторов, связанных как с эффективностью операций, так и с улучшением качества обслуживания клиентов.

Основная проблема, с которой сталкиваются большинство фитнес-центров, это необходимость одновременного управления различными аспектами деятельности: расписание занятий, управление абонементами, учет посещений, управление тренерами, учет финансовых операций и аналитика. Ручное ведение этих процессов приводит к ошибкам, задержкам в обслуживании, потере данных и снижению удовлетворенности клиентов.

Целью настоящей курсовой работы является разработка информационной системы управления фитнес-центром под названием FitTime, которая должна автоматизировать основные бизнес-процессы фитнес-центра и обеспечить эффективное управление ресурсами, клиентами и финансами.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области фитнес-центров и изучить типичные бизнес-процессы, применяемые в отрасли;

2. Спроектировать архитектуру информационной системы, включая проектирование структуры базы данных и взаимодействие компонентов;

3. Разработать программные модули для реализации основной функциональности системы, включая авторизацию, управление клиентами, управление абонементами, расписание занятий, учет посещений и генерацию отчетов;

4. Осуществить тестирование разработанной системы и проверку ее соответствия требованиям;

5. Подготовить документацию и рекомендации по внедрению системы в фитнес-центр.

Объектом исследования является информационная система управления фитнес-центром FitTime, предназначенная для автоматизации основных операционных процессов.

Предметом исследования являются методы проектирования и разработки информационных систем управления фитнес-центрами, включая анализ предметной области, проектирование архитектуры системы и разработку основных функциональных модулей.

При разработке системы использовались следующие методики и инструменты:

- Архитектурный паттерн MVVM (Model-View-ViewModel) для обеспечения разделения ответственности и облегчения тестирования кода;

- Объектно-ориентированное проектирование и программирование для создания переиспользуемых и поддерживаемых компонентов;
- Проектирование реляционной базы данных с нормализацией для обеспечения целостности и эффективности хранения данных;
- Методология работы с большими объемами данных с использованием ORM-фреймворков.

Технологический стек, используемый при разработке:

- Язык программирования C# версии 12 на платформе .NET 8 для обеспечения кроссплатформенности и производительности;
- WPF (Windows Presentation Foundation) для создания богатого пользовательского интерфейса настольного приложения;
- PostgreSQL 18 как надежная, масштабируемая и открытая система управления реляционной базой данных;
- Entity Framework Core 8 для объектно-реляционного отображения и удобной работы с данными;
- MVVM Community Toolkit для реализации паттерна MVVM с минимальной бойлерплейт-кодом;
- MaterialDesignInXaml для создания современного и привлекательного пользовательского интерфейса с поддержкой темной темы;
- BCrypt.Net-Next для криптографического хеширования паролей и защиты учетных данных;
- Serilog для структурированного логирования событий и отладки приложения;
- CsvHelper для экспорта данных в формат CSV для дальнейшего анализа в Excel и других инструментах.

Разработанная система предназначена для использования администраторами, менеджерами и тренерами фитнес-центров и должна обеспечивать удобство в использовании, надежность в работе и безопасность данных клиентов.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области и постановка задач

Фитнес-индустрия переживает период интенсивного роста и развития, особенно в условиях растущего спроса на услуги здоровья и благополучия. Однако многие фитнес-центры до сих пор используют устаревшие системы управления, основанные на бумажной документации или простых электронных таблицах. Это создает множество проблем: потеря данных, неправильные расчеты, задержки в обслуживании клиентов и невозможность проведения комплексного анализа бизнес-операций.

Типичные бизнес-процессы, осуществляемые в фитнес-центре, включают:

1. Регистрация и управление клиентами. Фитнес-центры должны вести полную информацию о каждом клиенте, включая личные данные, контактную информацию, историю членства и посещений. Это позволяет персоналу оказывать персонализированное обслуживание и отслеживать прогресс клиентов.

2. Управление типами абонементов и продажа членства. Фитнес-центры предлагают различные типы абонементов с разными сроками действия, количеством посещений и ценами. Необходимо эффективное управление этими предложениями и отслеживание продаж.

3. Планирование расписания занятий. Фитнес-центры проводят многочисленные групповые занятия в разные дни и время. Каждое занятие требует назначения тренера, выбора помещения, установления максимального количества участников и поддержания актуального расписания.

4. Управление тренерами. Тренеры являются ключевыми ресурсами для фитнес-центра. Необходимо управление их графиком, специализацией, квалификацией и взаимодействием с клиентами.

5. Учет посещений. Каждое посещение клиентом занятия или фитнес-центра должно быть зафиксировано для целей учета абонемента, аналитики и контроля качества.

6. Финансовое управление. Включает отслеживание продаж, доходов, расходов и генерацию финансовых отчетов.

7. Аналитика и отчетность. Руководство фитнес-центра нуждается в своевременных и точных отчетах о ключевых показателях эффективности (КПЭ), включая количество активных членов, популярность занятий, уровень посещаемости и тренды в продажах.

На текущем рынке существуют различные решения для управления фитнес-центрами. К наиболее известным относятся:

- 1С:Фитнес. Система на основе платформы 1С позволяет управлять основными бизнес-процессами. Преимущества: интеграция с 1С:Бухгалтерией, локализация для России. Недостатки: высокая стоимость лицензии, необходимость специалиста по 1С для внедрения и поддержки, относительно низкая скорость работы с большими объемами данных.

- Mobifitness. Облачная система с мобильным приложением. Преимущества: доступность из любого места, поддержка мобильных платформ. Недостатки: зависимость от интернета, ограниченная возможность кастомизации, вопросы по безопасности данных при облачном хранении.

- FitBase. Специализированное решение для фитнес-индустрии с расширенными возможностями аналитики. Преимущества: удобный интерфейс, хорошая поддержка. Недостатки: относительно высокая стоимость, ограничение на количество пользователей.

Несмотря на наличие готовых решений, разработка собственной информационной системы для конкретного фитнес-центра имеет ряд преимуществ:

- Полная кастомизация под специфические потребности бизнеса;
- Отсутствие абонентской платы, что экономит средства в долгосрочной перспективе;
- Полный контроль над данными и безопасностью;
- Возможность интеграции с другими системами, используемыми в центре;
- Простота добавления новых функций по мере развития бизнеса.

Функциональные требования к системе FitTime включают:

1. Авторизация и управление ролями. Система должна поддерживать различные роли пользователей (Администратор, Менеджер, Тренер) с соответствующим разделением прав доступа. Реализация должна обеспечивать безопасное хранение паролей и механизм блокировки аккаунта при множественных неудачных попытках входа.

2. Управление клиентами. CRUD-операции для клиентов, поиск и фильтрация по различным критериям, ведение истории членства и посещений.

3. Управление типами абонементов и продажа членства. Возможность создания и редактирования типов абонементов, регистрация продажи новых абонементов, отслеживание статуса и срока действия.

4. Расписание занятий. Еженедельный календарь с возможностью добавления, редактирования и удаления занятий. Каждое занятие содержит информацию о типе, тренере, времени, помещении и максимальном числе участников.

5. Учет посещений. Ведение журнала посещений с отслеживанием статуса (Посетил, Не пришел, Отменено), связь с членством клиента и вычет визитов из лимита.

6. Генерация отчетов. Создание различных аналитических отчетов: по посещаемости, по продажам, по популярности занятий, с возможностью экспорта в CSV.

7. Панель управления (Dashboard). Отображение ключевых показателей эффективности (КПЭ), включая количество активных членов, планы на текущий день, продажи за месяц и скоро истекающие абонементы.

8. Экспорт данных. Возможность экспорта различных отчетов в формат CSV для дальнейшего анализа в Excel.

Нефункциональные требования:

1. Производительность. Время отклика для основных операций должно быть менее 2 секунд при использовании на 100+ одновременных пользователей.

2. Безопасность. Использование криптографического хеширования для паролей (BCrypt с коэффициентом 11), ролевой контроль доступа, защита от SQL-инъекций благодаря использованию параметризованных запросов.

3. Удобство использования. Современный и интуитивный интерфейс на основе Material Design с поддержкой темной темы, удобная навигация, минимальное количество кликов для выполнения типичных операций.

4. Надежность. Система должна обеспечивать целостность данных, безопасное восстановление при сбоях, регулярное резервное копирование.

5. Масштабируемость. Архитектура должна позволять добавление новых функций и увеличение объемов данных без необходимости полного переписывания системы.

1.2 Моделирование и проектирование системы

Проектирование системы начинается с моделирования различных аспектов, включая процессы, взаимодействие компонентов и структуру данных. Для этого использованы различные диаграммы UML, которые позволяют визуализировать архитектуру и поведение системы.

Первый этап моделирования включает построение диаграммы вариантов использования (Use Case diagram), которая показывает основные функции системы и взаимодействие различных типов пользователей (акторов) с системой. В системе FitTime определены три основных роли: Администратор, Менеджер и Тренер, каждый из которых взаимодействует с различными компонентами системы. Администратор имеет полный доступ ко всем функциям, включая управление пользователями и конфигурацию системы. Менеджер может управлять клиентами, абонементами, расписанием и просматривать отчеты. Тренер может просматривать свое расписание, управлять посещениями своих занятий и просматривать информацию о клиентах, посещающих его занятия.

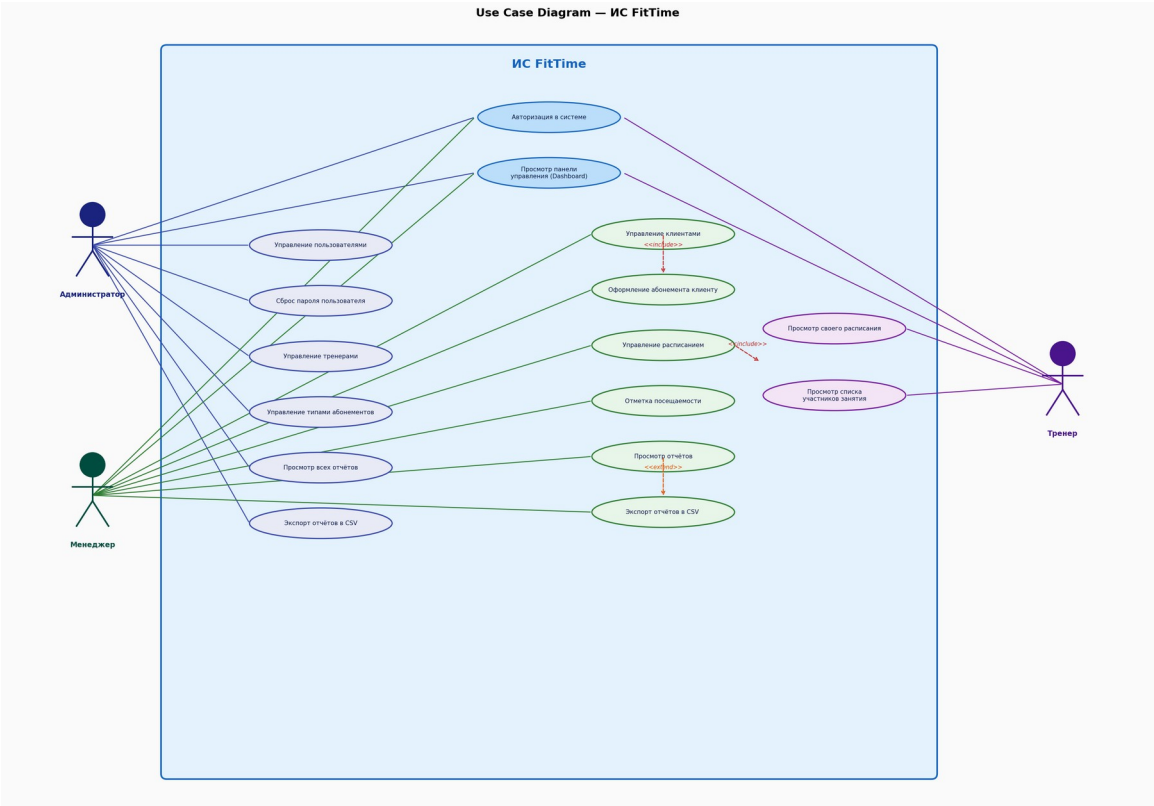

Рисунок 1 — Диаграмма вариантов использования ИС FitTime

После определения вариантов использования следует рассмотрение процесса авторизации. Диаграмма последовательности (Sequence diagram) для процесса авторизации показывает взаимодействие компонентов системы при входе пользователя. Пользователь вводит учетные данные в окне LoginWindow, которое направляет запрос на LoginViewModel. ViewModel осуществляет поиск пользователя в базе данных через FitTimeDbContext и проверяет пароль с использованием алгоритма BCrypt. При успешной аутентификации информация о текущем пользователе сохраняется в ICurrentUserService, и приложение переходит к главному окну MainWindow.

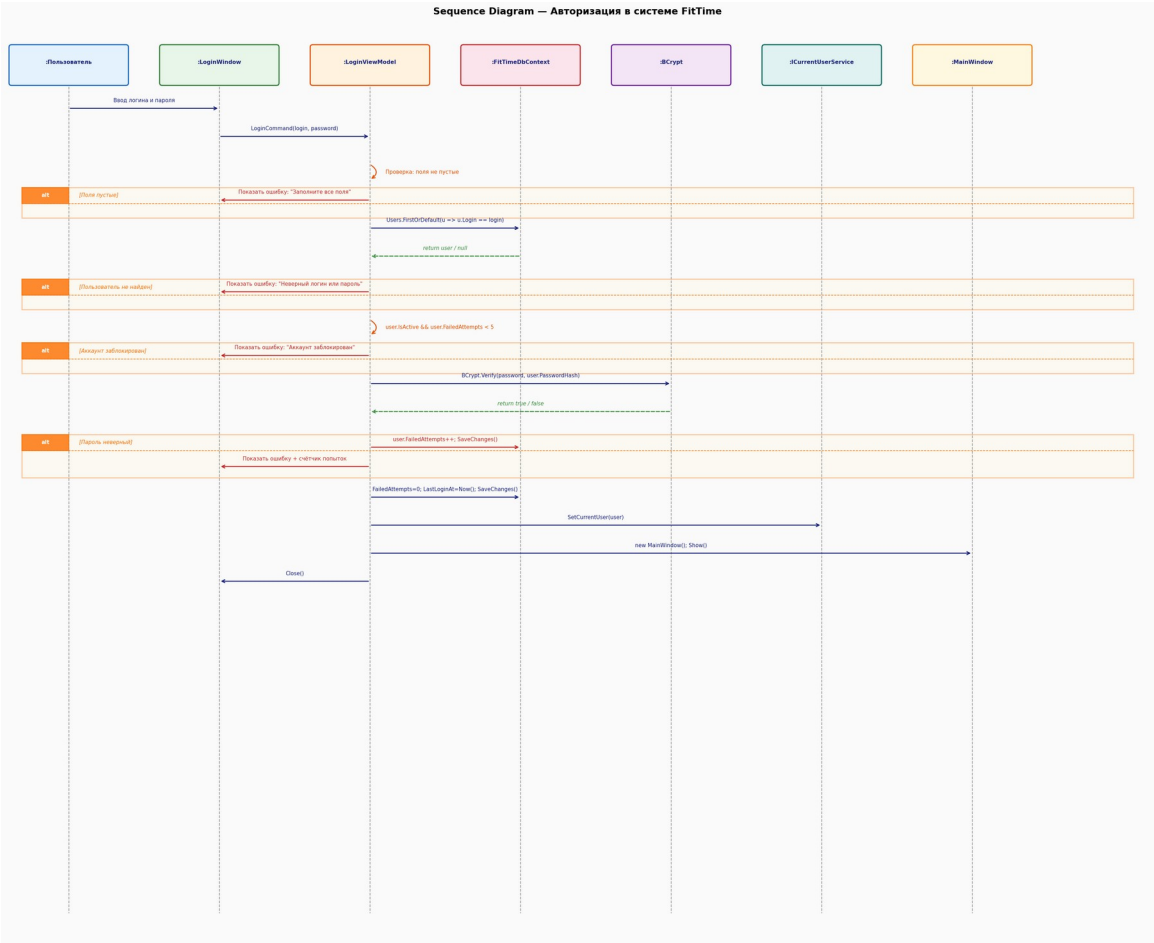

Рисунок 2 — Диаграмма последовательности процесса авторизации

Диаграмма деятельности (Activity diagram) для процесса оформления абонемента демонстрирует пошаговый ход выполнения бизнес-процесса от начала до конца. Процесс начинается с выбора клиента, затем пользователь выбирает тип абонемента из доступных опций. После выбора типа системе требуется вычислить даты начала и окончания членства на основе выбранного типа. Пользователь вводит или подтверждает цену, и система проверяет наличие активного абонемента у клиента. Если активный абонемент существует, система предупреждает об этом. Затем система сохраняет новое членство в базе данных и создает запись о продаже. Процесс завершается уведомлением пользователю об успешном оформлении.

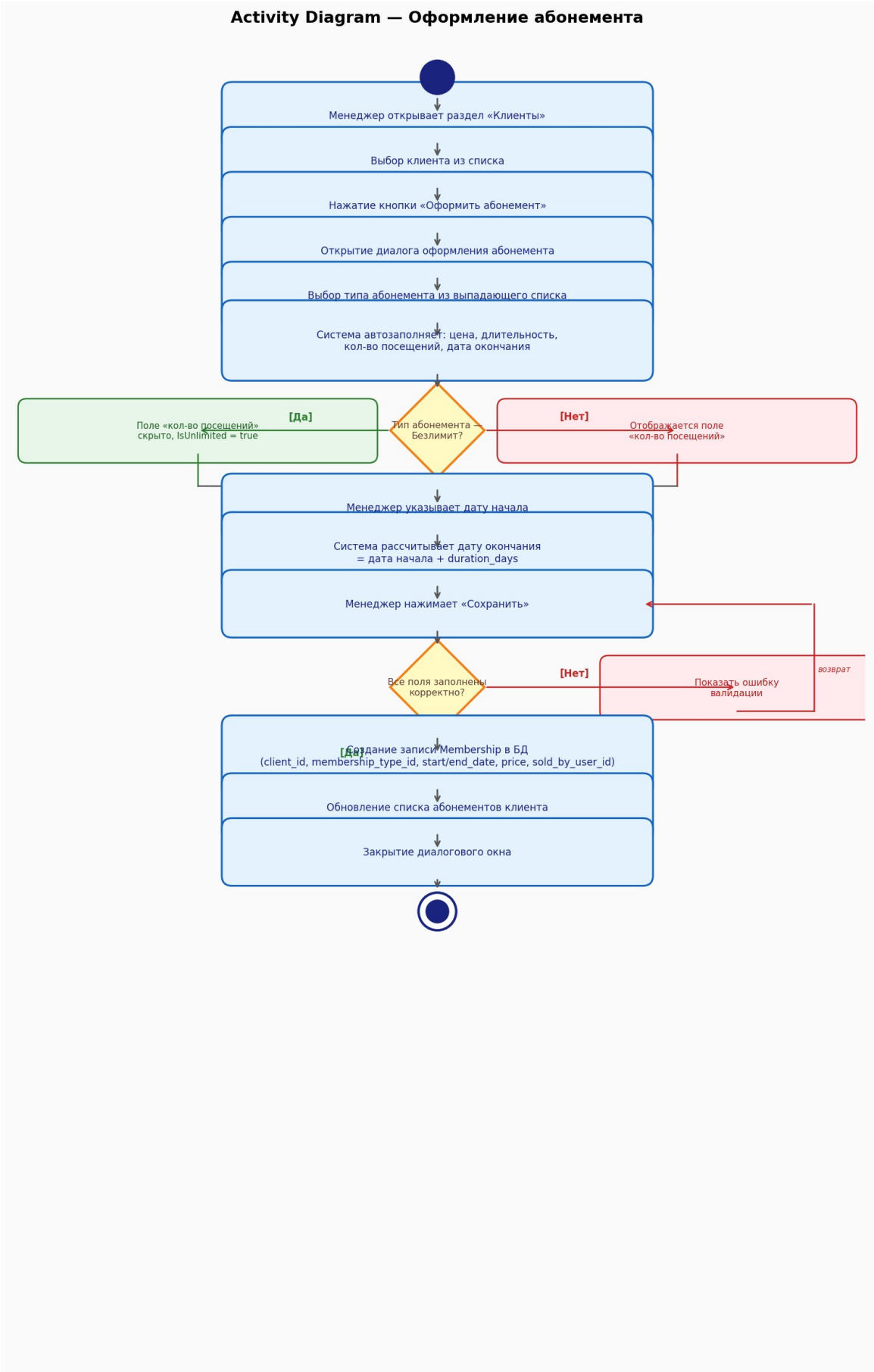
Activity Diagram — Оформление абонемента
Менеджер открывает раздел «Клиенты»
Выбор клиента из списка
Нажатие кнопки «Оформить абонемент»
Открытие диалога оформления абонемента
Выбор типа абонемента из выпадающего списка
Система автозаполняет: цена, длительность, кол-во посещений, дата окончания
Тип абонемента — Безлимит?
[Да]
Поле «кол-во посещений» скрыто, IsUnlimited = true
[Нет]
Отображается поле «кол-во посещений»
Менеджер указывает дату начала
Система рассчитывает дату окончания = дата начала + duration_days
Менеджер нажимает «Сохранить»
Все поля заполнены корректно?
[Нет]
Показать ошибку валидации
возврат
[Да]
Создание записи Membership в БД (client_id, membership_type_id, start/end_date, price, sold_by_user_id)
Обновление списка абонементов клиента
Закрытие диалогового окна

Рисунок 3 — Диаграмма деятельности процесса оформления абонемента

Проектирование базы данных осуществляется с использованием ER-диаграммы (Entity-Relationship diagram), которая определяет все таблицы, их атрибуты и связи. В системе FitTime используется 9 основных таблиц и 2 представления для оптимизации часто используемых запросов.

Таблица roles хранит информацию о доступных ролях в системе: Администратор, Менеджер, Тренер.

Таблица users содержит информацию обо всех системных пользователях, включая их учетные данные, роль, контактную информацию и специализацию (для тренеров). Пароли хранятся в виде хеша BCrypt.

Таблица clients содержит информацию о клиентах фитнес-центра: ФИО, дату рождения, контактные данные, примечания и статус активности.

Таблица membership_types определяет различные типы абонементов, которые может предложить фитнес-центр, включая название, длительность в днях, количество визитов, цену и статус архивирования.

Таблица memberships хранит информацию о каждом активном или завершенном абонементе клиента, включая ссылку на клиента и тип абонемента, дату начала и окончания, количество оставшихся визитов и цену.

Таблица class_types определяет типы фитнес-занятий, которые проводятся в центре (например, Йога, Пилатес, Кроссфит и т.д.), каждый тип имеет описание и цвет для визуализации.

Таблица classes содержит информацию о конкретных занятиях, включая тип занятия, назначенного тренера, время проведения, помещение, максимальное количество участников и статус.

Таблица class_enrollments регистрирует регистрацию клиентов на конкретные занятия, с указанием клиента, занятия, абонемента и статуса регистрации.

Таблица attendance ведет журнал посещений клиентов, регистрируя каждый визит с указанием времени, статуса (Присутствовал, Не пришел, Отменено), пользователя, который зарегистрировал визит, и абонемента.

Для оптимизации часто используемых запросов созданы два представления (views): vw_active_memberships для быстрого получения активных членств клиентов и vw_class_schedule для отображения расписания занятий с полной информацией об типе, тренере и помещении.

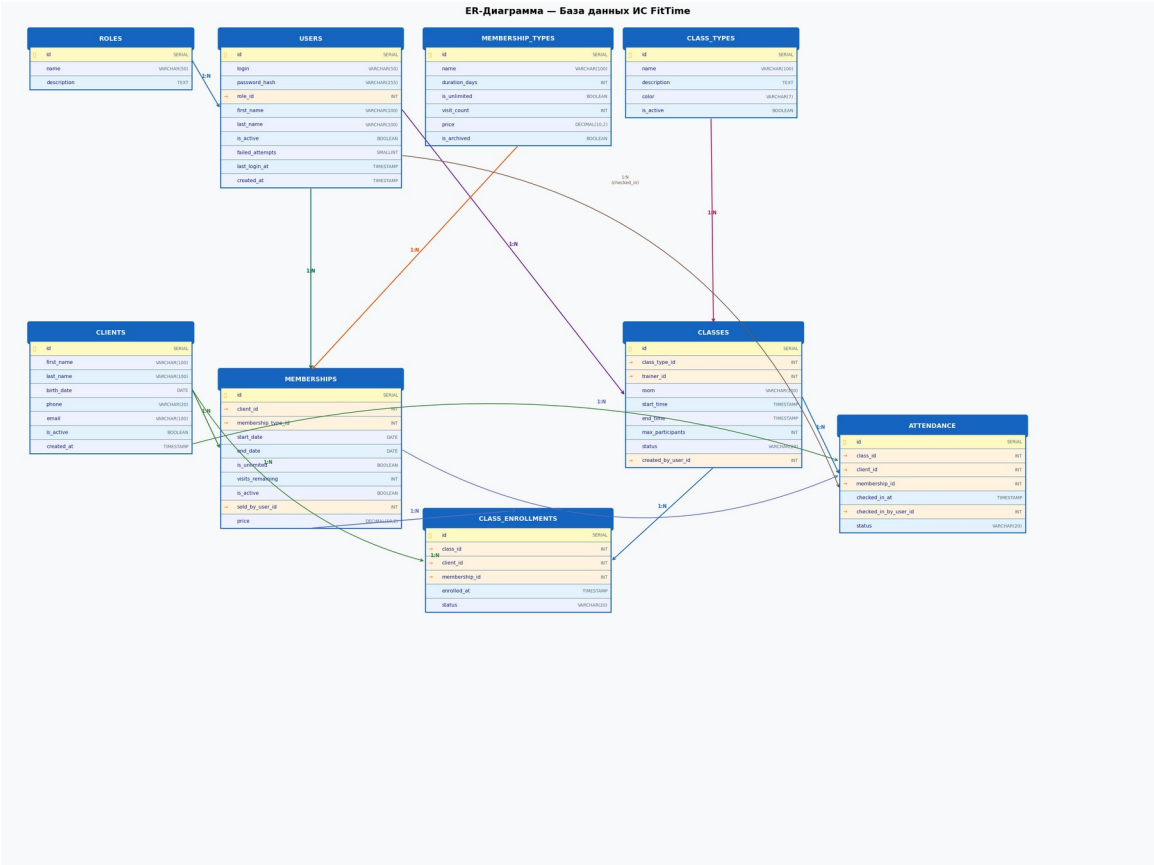

Рисунок 4 — ER-диаграмма базы данных FitTime

Диаграмма классов (Class diagram) показывает архитектуру приложения и организацию кода в четыре основных пакета.

Пакет Models содержит все модели данных, которые отображают таблицы базы данных в объектно-ориентированное представление. Каждый класс в этом пакете соответствует сущности в базе данных.

Пакет ViewModels содержит классы ViewModels, которые реализуют паттерн MVVM. Каждый ViewModel содержит логику представления и управляет взаимодействием между View и Services. Примеры включают LoginViewModel для управления процессом входа, ClientsViewModel для управления списком клиентов, MembershipsViewModel для управления абонементами и другие.

Пакет Services содержит сервисы, которые предоставляют различные функциональности для приложения. К ним относятся INavigationService для навигации между экранами, ICurrentUserService для отслеживания текущего пользователя, IDialogService для отображения диалогов подтверждения, и другие сервисы бизнес-логики.

Пакет Data содержит FitTimeDbContext, который является контекстом базы данных Entity Framework Core и определяет все DbSet для каждой модели, а также конфигурацию моделей через Fluent API.

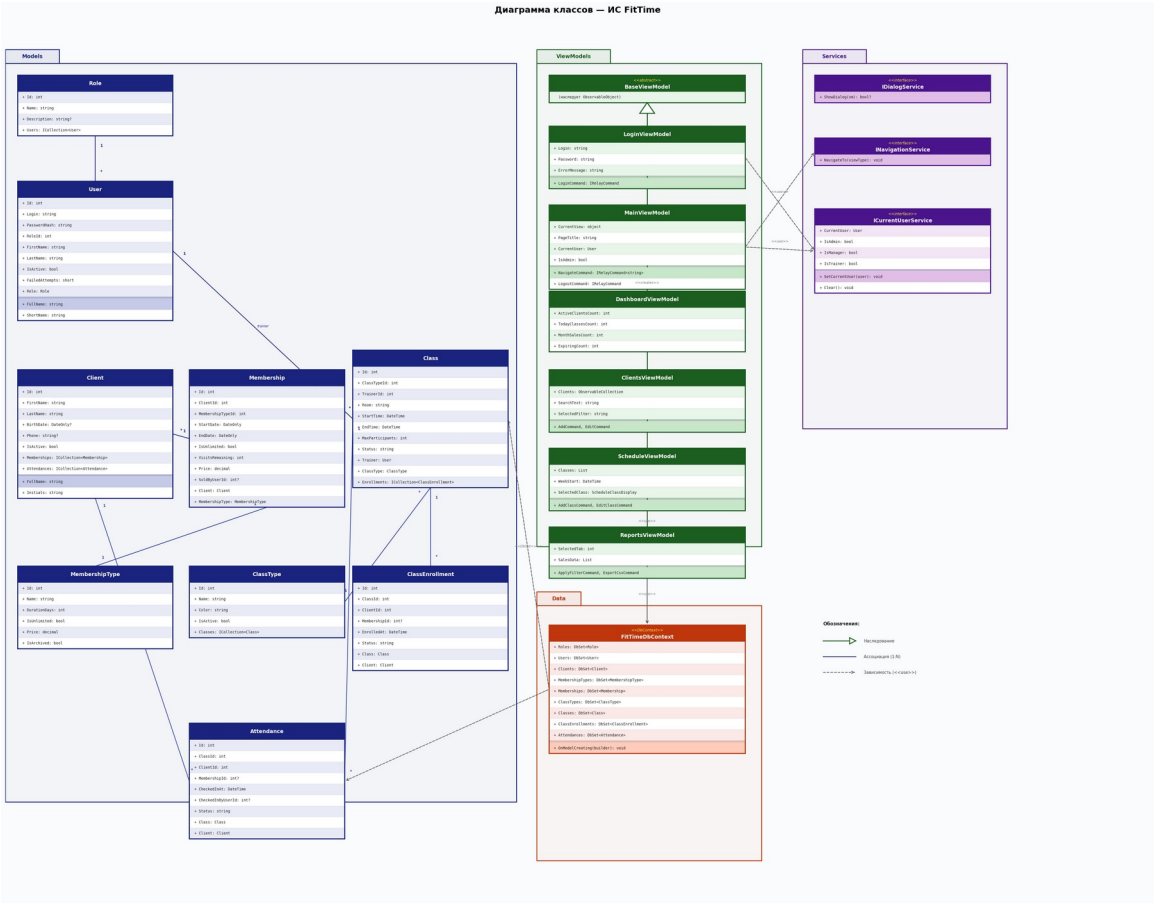

Рисунок 5 — Диаграмма классов приложения FitTime

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Разработка базы данных

Разработка базы данных системы FitTime осуществляется на основе PostgreSQL версии 18, которая была выбрана за ее надежность, масштабируемость, открытый исходный код и отличную поддержку для корпоративных приложений. Для взаимодействия с базой данных из приложения используется Entity Framework Core 8, мощный ORM-фреймворк, который позволяет писать запросы на языке C# LINQ вместо SQL.

Архитектура базы данных основана на принципах нормализации, что обеспечивает минимизацию дублирования данных и сохранение целостности информации. Все таблицы используют snake_case соглашение для имен столбцов, что является стандартной практикой в PostgreSQL.

Таблица roles содержит следующие поля:
- id (целое число, первичный ключ) - уникальный идентификатор роли
- name (строка, уникальная) - название роли
- description (текст) - описание прав и ответственности роли

Таблица users содержит:
- id (целое число, первичный ключ) - уникальный идентификатор пользователя
- login (строка, уникальная) - учетное имя для входа в систему
- password_hash (строка) - хеш пароля, вычисленный алгоритмом BCrypt
- role_id (целое число, внешний ключ) - ссылка на роль пользователя
- first_name (строка) - имя пользователя
- last_name (строка) - фамилия пользователя
- patronymic (строка) - отчество пользователя
- phone (строка) - телефонный номер для связи
- email (строка) - адрес электронной почты
- specialization (строка) - специализация для тренеров
- is_active (логический флаг) - статус активности аккаунта
- failed_attempts (целое число) - счетчик неудачных попыток входа
- last_login_at (дата/время) - время последнего успешного входа
- created_at (дата/время) - время создания учетной записи

Таблица clients содержит:
- id (целое число, первичный ключ)
- first_name (строка) - имя клиента
- last_name (строка) - фамилия клиента
- patronymic (строка) - отчество клиента
- birth_date (дата) - дата рождения
- phone (строка) - номер телефона для контакта
- email (строка) - адрес электронной почты
- notes (текст) - дополнительные примечания о клиенте
- is_active (логический флаг) - статус активности клиента
- created_at (дата/время) - дата регистрации в центре
- updated_at (дата/время) - дата последнего обновления информации

Таблица membership_types содержит:
- id (целое число, первичный ключ)
- name (строка) - название типа абонемента (например "Эконом месяц")
- duration_days (целое число) - количество дней действия абонемента
- is_unlimited (логический флаг) - является ли абонемент безлимитным по посещениям
- visit_count (целое число) - максимальное количество посещений для ограниченных абонементов
- price (денежное значение) - стоимость абонемента
- description (текст) - описание включаемых услуг
- is_archived (логический флаг) - архивирована ли тип абонемента
- created_at (дата/время) - дата создания типа

Таблица memberships содержит:
- id (целое число, первичный ключ)
- client_id (целое число, внешний ключ) - ссылка на клиента
- membership_type_id (целое число, внешний ключ) - ссылка на тип абонемента
- start_date (дата) - дата начала действия абонемента
- end_date (дата) - дата окончания действия абонемента
- is_unlimited (логический флаг) - является ли это членство безлимитным
- visits_remaining (целое число) - количество оставшихся визитов
- is_active (логический флаг) - активно ли это членство
- sold_by_user_id (целое число, внешний ключ) - пользователь, оформивший абонемент
- price (денежное значение) - фактическая цена, по которой был куплен абонемент
- notes (текст) - примечания о членстве
- created_at (дата/время) - дата оформления абонемента

Таблица class_types содержит:
- id (целое число, первичный ключ)
- name (строка) - название типа занятия
- description (текст) - описание занятия
- color (строка) - цветовой код для визуализации
- is_active (логический флаг) - активен ли данный тип

Таблица classes содержит:
- id (целое число, первичный ключ)
- class_type_id (целое число, внешний ключ) - ссылка на тип занятия
- trainer_id (целое число, внешний ключ) - ссылка на назначенного тренера
- room (строка) - номер или название помещения
- start_time (дата/время) - время начала занятия
- end_time (дата/время) - время окончания занятия
- max_participants (целое число) - максимальное количество участников
- status (строка) - статус занятия (Запланировано, Проводится, Завершено, Отменено)
- notes (текст) - примечания о занятии
- created_by_user_id (целое число, внешний ключ) - пользователь, создавший запись о занятии
- created_at (дата/время) - дата создания записи

Таблица class_enrollments содержит:
- id (целое число, первичный ключ)
- class_id (целое число, внешний ключ) - ссылка на занятие

- client_id (целое число, внешний ключ) - ссылка на клиента
- membership_id (целое число, внешний ключ) - ссылка на абонемент
- enrolled_at (дата/время) - время регистрации на занятие
- status (строка) - статус регистрации (Зарегистрирован, Подтвержден, Отменен)

Таблица attendance содержит:
- id (целое число, первичный ключ)
- class_id (целое число, внешний ключ) - ссылка на занятие
- client_id (целое число, внешний ключ) - ссылка на клиента
- membership_id (целое число, внешний ключ) - ссылка на использованный абонемент
- checked_in_at (дата/время) - время регистрации посещения
- checked_in_by_user_id (целое число, внешний ключ) - пользователь, зарегистрировавший посещение
- status (строка) - статус посещения (Присутствовал, Не пришел, Отменено)
- notes (текст) - примечания о посещении

Система использует несколько индексов для оптимизации производительности часто используемых запросов:
- idx_clients_last_name - индекс по фамилии клиента для быстрого поиска
- idx_clients_phone - индекс по телефону для быстрого поиска по номеру
- idx_clients_email - индекс по электронной почте для быстрого поиска по email
- Уникальный индекс на сочетание (class_id, client_id) в таблице class_enrollments для предотвращения дублирования регистраций

Конфигурация моделей в Entity Framework Core выполняется с помощью Fluent API, что позволяет определить все аспекты отображения в одном месте, включая имена таблиц, столбцы, типы данных, ограничения и связи.

2.2 Администрирование базы данных

Администрирование базы данных включает различные операции, необходимые для обеспечения надежности, безопасности и производительности системы.

Стратегия резервного копирования базы данных основана на использовании команды pg_dump из PostgreSQL. Ежедневно выполняются автоматические резервные копии полной базы данных, которые сохраняются на отдельном диске с датой в имени файла. Это обеспечивает возможность восстановления данных в случае сбоя или потери.

Управление доступом к базе данных осуществляется на уровне ролей PostgreSQL. Для работы приложения создана отдельная роль приложения с ограниченными правами: возможность чтения и записи в основные таблицы, но без возможности создания или удаления таблиц. Административная роль имеет полные права для выполнения операций обслуживания.

Управление подключениями к базе данных осуществляется через встроенное в Entity Framework Core управление пулом соединений. Это предотвращает истощение ресурсов системы при высокой нагрузке и обеспечивает эффективное переиспользование соединений.

Мониторинг базы данных и приложения реализуется с использованием Serilog - фреймворка

структурированного логирования. Все операции логируются с указанием уровня серьезности (Debug, Information, Warning, Error), временем события и релевантным контекстом. Логи сохраняются в файлы для последующего анализа.

Обеспечение целостности данных достигается через использование ограничений целостности на уровне базы данных:
- Внешние ключи (foreign keys) обеспечивают, что ссылки на другие таблицы всегда указывают на существующие записи
- Уникальные ограничения предотвращают дублирование критически важных данных (логины пользователей, например)
- Проверочные ограничения (check constraints) гарантируют соответствие данных определенным условиям

Все операции с базой данных выполняются в контексте транзакций, что гарантирует атомарность операций - либо вся операция выполняется успешно, либо откатывается полностью.

Индексы регулярно анализируются и перестраиваются для поддержания оптимальной производительности. Система отслеживает размер базы данных и используемое дисковое пространство для планирования емкости.

2.3 Разработка программных модулей

Практическая реализация системы FitTime состоит из семи основных программных модулей, каждый из которых обеспечивает определенную функциональность. Далее описывается реализация каждого модуля с демонстрацией пользовательского интерфейса и ключевых функций.

Модуль 1: Авторизация (Authentication)

Модуль авторизации является критически важным компонентом системы, обеспечивающим безопасный доступ к приложению. При запуске приложения пользователь видит окно входа, где необходимо ввести учетные данные.

Процесс авторизации включает следующие шаги:

1. Пользователь вводит свой логин и пароль в соответствующих полях.

2. При нажатии кнопки "Вход" приложение проверяет, что оба поля заполнены.

3. Система осуществляет поиск пользователя в базе данных по введенному логину.

4. Если пользователь найден, проверяется статус активности его учетной записи.

5. Пароль проверяется с использованием алгоритма BCrypt. Введенный пароль хешируется и сравнивается с сохраненным хешем.

6. При неверном пароле счетчик неудачных попыток входа увеличивается на единицу. После 5 неудачных попыток учетная запись блокируется.

7. При успешной аутентификации счетчик неудачных попыток сбрасывается, время последнего входа обновляется в базе данных, и приложение переходит к главному окну.

Использование BCrypt обеспечивает криптографическую стойкость: пароли невозможно восстановить из хешей, а каждый хеш уникален благодаря использованию соли.

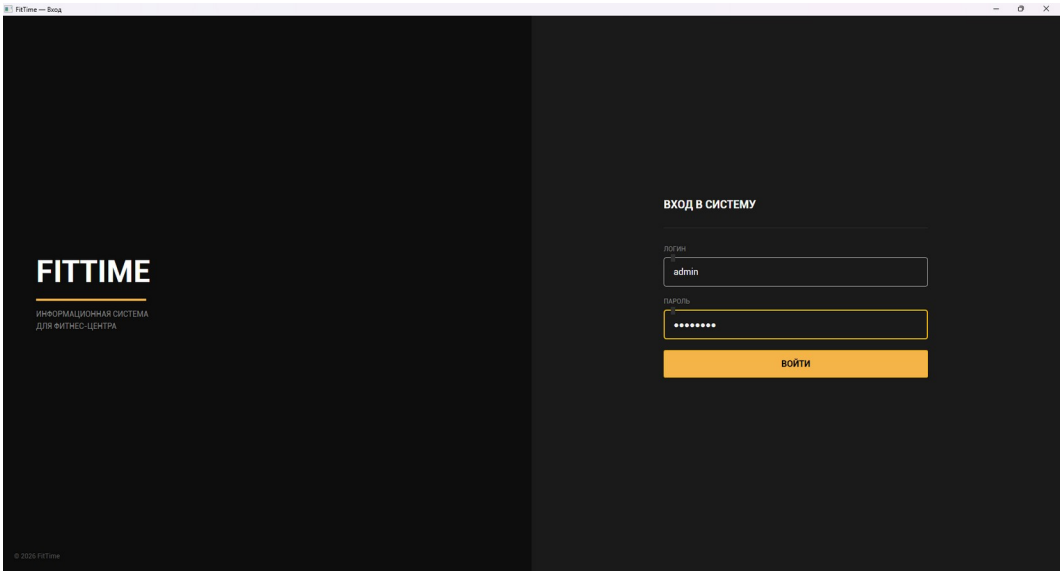

Рисунок 6 — Окно авторизации

Модуль 2: Панель управления (Dashboard)

Панель управления (Dashboard) предоставляет администратору и менеджеру быстрый обзор ключевых показателей эффективности фитнес-центра. Это помогает принимать информированные управленческие решения.

На панели управления отображаются следующие элементы:

1. Четыре карточки с ключевыми показателями:
   - Активные клиенты: общее количество клиентов с активными абонементами
   - Занятия сегодня: количество запланированных групповых занятий на текущий день
   - Продажи за месяц: общая сумма доходов от продажи абонементов в текущем месяце
   - Скоро истекают: количество абонементов, которые истекают в течение следующих 7 дней

2. График еженедельной посещаемости: столбчатая диаграмма, показывающая количество посещений по дням недели.

3. Список занятий на сегодня: таблица с информацией о каждом занятии, включая тип, тренера, время, помещение и статус.

4. Недавние продажи: список последних оформленных абонементов с информацией о клиенте, типе абонемента и цене.

5. Скоро истекающие абонементы: список абонементов, которые истекут в ближайшее время, позволяющий менеджерам вовремя напомнить клиентам о продлении.

Все данные на панели обновляются при загрузке и могут быть обновлены вручную нажатием кнопки "Обновить".

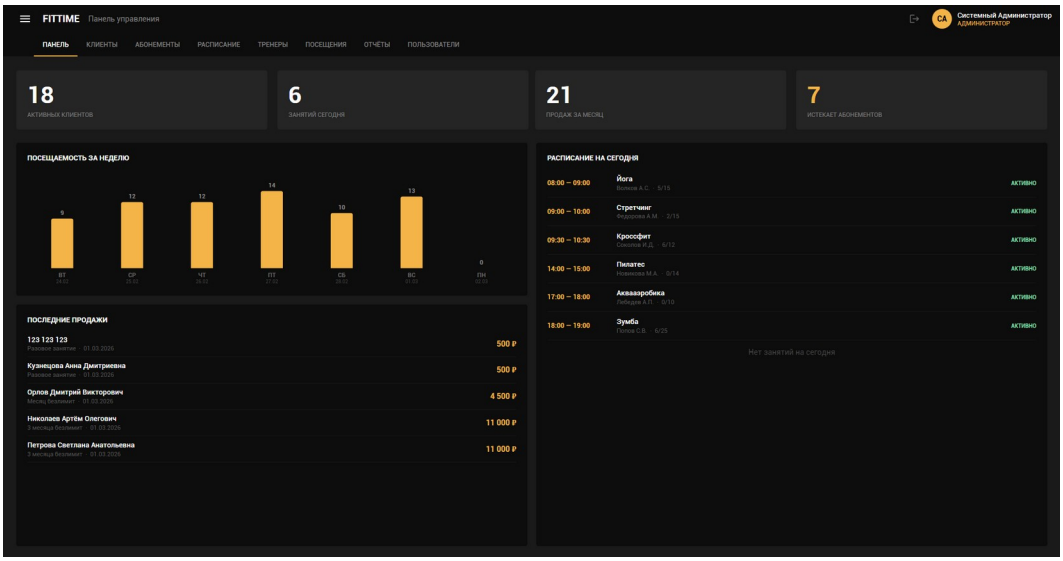

Рисунок 7 — Панель управления (Dashboard)

Модуль 3: Управление клиентами

Модуль управления клиентами предоставляет полный набор инструментов для работы с базой данных клиентов фитнес-центра. Менеджеры и администраторы могут создавать новые записи клиентов, редактировать существующие данные, осуществлять поиск и фильтрацию.

Основное окно модуля содержит таблицу со списком клиентов. Для каждого клиента отображаются: полное имя, номер телефона, адрес электронной почты, статус членства и дата регистрации.

Над таблицей находится строка поиска, которая позволяет быстро найти клиента по фамилии, имени, номеру телефона или адресу электронной почты. Поиск выполняется в реальном времени при вводе текста.

Рядом со строкой поиска расположены кнопки фильтрации:
- "Все" - показывает всех клиентов
- "Активные" - только клиенты с действующими абонементами
- "Просроченные" - клиенты, у которых абонемент истек, но запись еще не архивирована
- "Без аб." - клиенты без активного абонемента

Статус членства отображается цветом: зеленый для активных, оранжевый для близких к истечению (менее 7 дней), красный для просроченных.

При выборе клиента в таблице можно нажать на кнопку "Профиль" для просмотра подробной информации о клиенте.

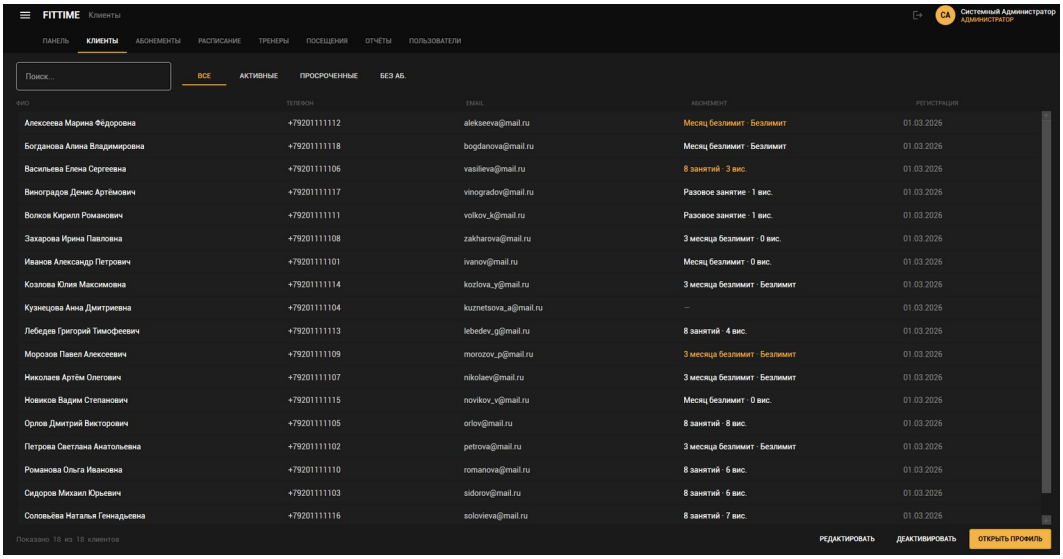

Рисунок 8 — Список клиентов

При открытии профиля клиента отображается подробная информация, организованная в несколько вкладок.

Вкладка "Личные данные" содержит информацию о клиенте: полное имя, дату рождения, контактные данные (телефон, email), дату регистрации в центре и примечания. Все поля можно редактировать.

Вкладка "Абонементы" показывает историю всех абонементов клиента, как активных, так и истекших. Для каждого абонемента отображаются: тип, дата начала, дата окончания, количество оставшихся визитов и статус. Отсюда можно оформить новый абонемент.

Вкладка "История посещений" содержит список всех посещений клиентом занятий с информацией о дате, времени, названии занятия, тренере и статусе посещения.

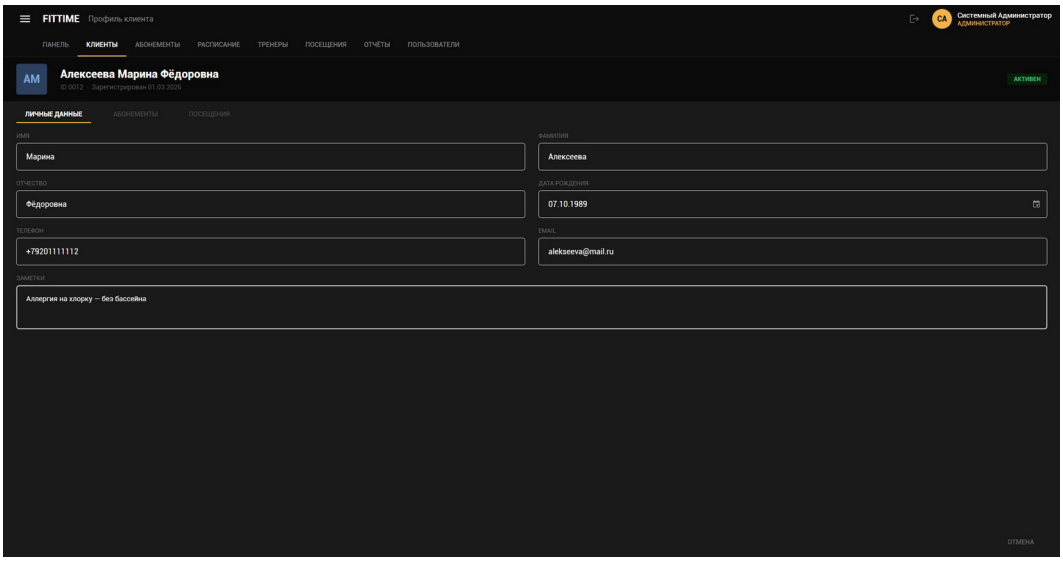

Рисунок 9 — Профиль клиента

Модуль 4: Управление абонементами

Управление абонементами - одна из ключевых функций фитнес-центра. Этот модуль позволяет определять типы абонементов и управлять их продажей.

Первый экран модуля показывает список доступных типов абонементов. Для каждого типа отображаются: название, количество дней действия (или отметка о безлимитности по времени), количество включаемых визитов (или "безлимит"), цена и статус (активный или архивированный).

Администратор может создавать новые типы абонементов, редактировать существующие и архивировать неиспользуемые. При создании типа абонемента указываются:
- Название (например, "Месячный безлимит")
- Количество дней действия
- Количество включаемых визитов (или отметка о безлимитности)
- Цена
- Описание включаемых услуг

После выбора типа абонемента можно нажать кнопку "Оформить абонемент", которая открывает диалог продажи.

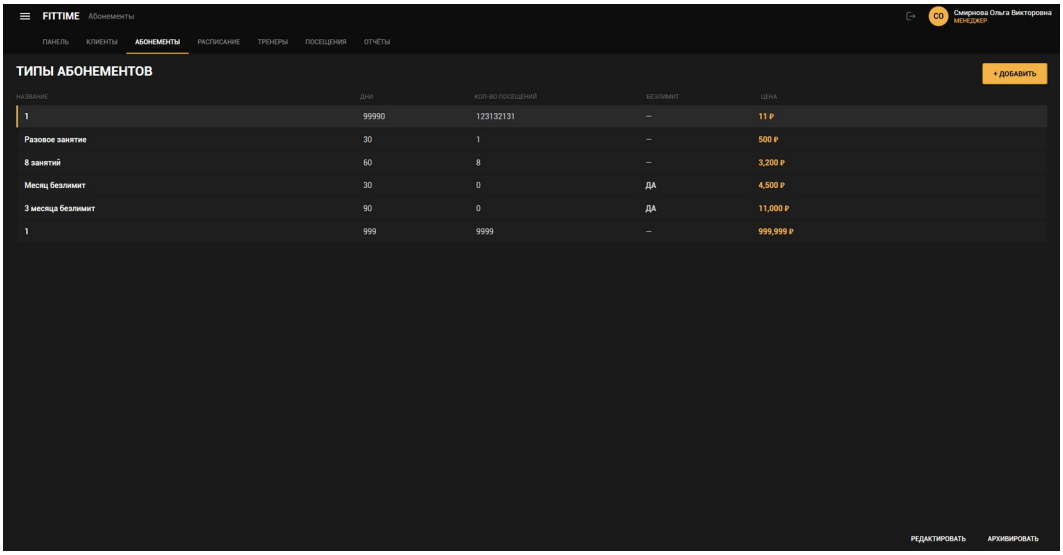

Рисунок 10 — Типы абонементов

Диалог оформления абонемента содержит следующие поля:

1. Выбор клиента - раскрывающееся меню с поиском, позволяющее выбрать клиента из базы данных.

2. Выбор типа абонемента - содержит информацию о выбранном типе абонемента.

3. Дата начала - может быть установлена как текущий день или выбран другой день.

4. Дата окончания - автоматически вычисляется на основе даты начала и продолжительности абонемента.

5. Цена - изначально заполняется ценой из типа абонемента, но может быть отредактирована

(например, при применении скидки).

6. Примечания - дополнительная информация об абонементе.

При нажатии кнопки "Сохранить" система проверяет, нет ли у клиента уже активного абонемента того же типа, и если все данные корректны, создает новую запись в базе данных.

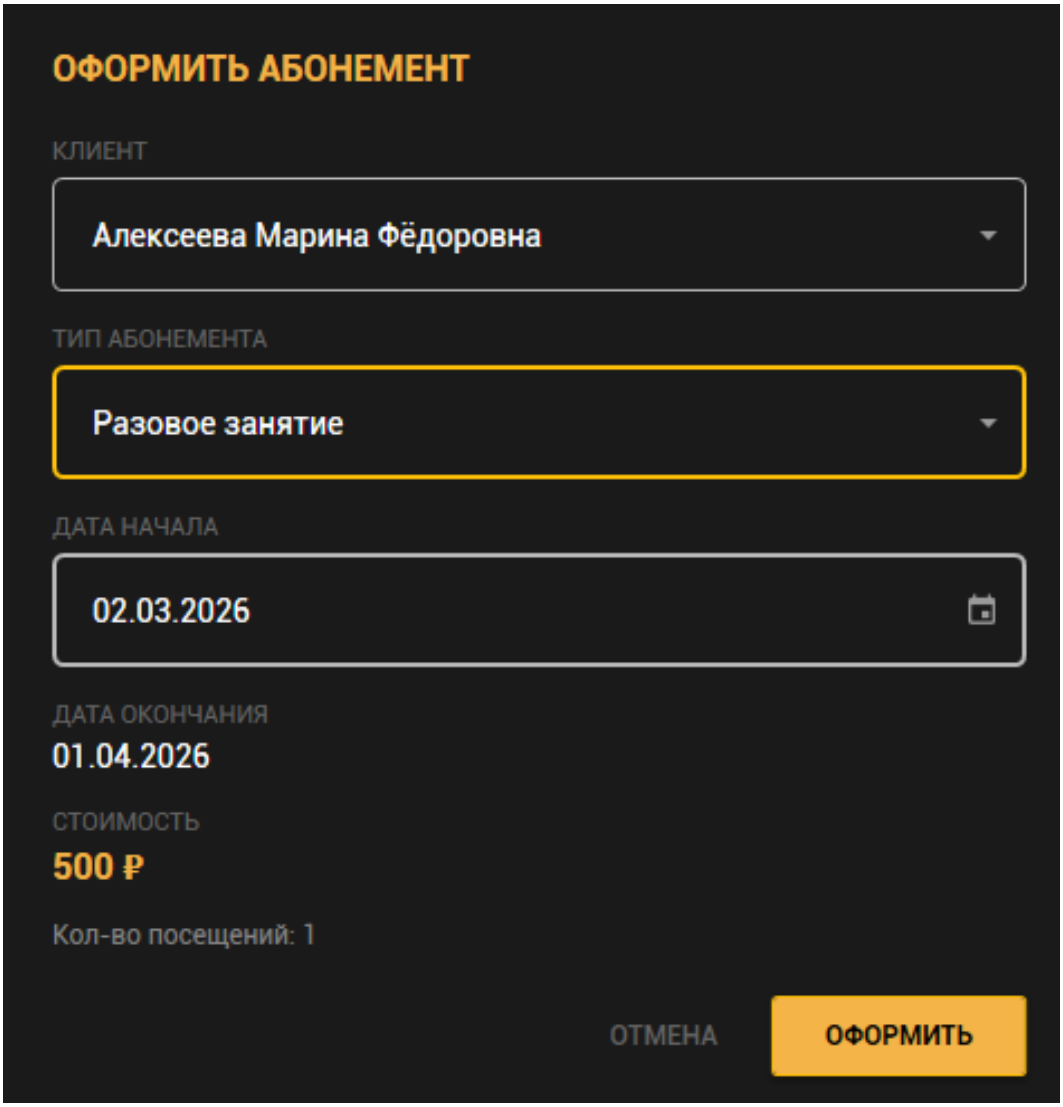

Рисунок 11 — Диалог оформления абонемента

Модуль 5: Расписание занятий

Модуль расписания позволяет управлять календарем групповых занятий. Основной вид представляет еженедельный календарь с отображением всех занятий.

Календарь организован в виде таблицы, где по вертикали расположены дни недели, а по горизонтали - часовые слоты времени. Каждое занятие отображается как блок на календаре, окрашенный в цвет типа занятия. На блоке отображаются основные сведения о занятии: название типа, имя тренера и время.

Над календарем находятся элементы управления:
- Кнопка "Предыдущая неделя" и "Следующая неделя" для навигации
- Отображение текущей недели
- Кнопка "Сегодня" для быстрого возврата на текущую неделю

Цветовое кодирование типов занятий облегчает визуальное восприятие расписания. Например, занятия йогой может отображаться синим цветом, занятия пилатесом - зеленым, кроссфит - оранжевым и т.д.

Администраторы и менеджеры могут добавлять новые занятия, редактировать существующие и удалять отмененные занятия путем нажатия соответствующих кнопок или двойного щелчка на блоке занятия.

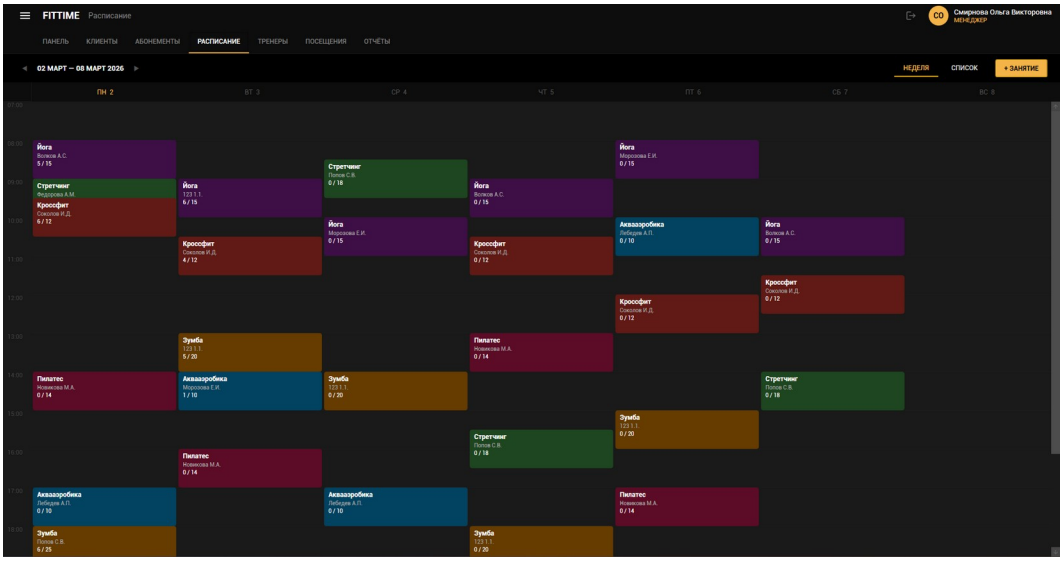

Рисунок 12 — Расписание в режиме недели

При щелчке на занятие открывается карточка с подробной информацией о занятии:

- Название типа занятия с цветовой меткой
- Тренер, проводящий занятие, с возможностью быстрого переключения между тренерами
- Время проведения (начало и конец)
- Помещение, где проводится занятие
- Максимальное количество участников
- Текущее количество зарегистрированных участников
- Статус занятия (Запланировано, Проводится, Завершено, Отменено)
- Примечания о занятии

На карточке присутствуют кнопки для редактирования информации о занятии, просмотра списка зарегистрированных клиентов и отмены занятия.

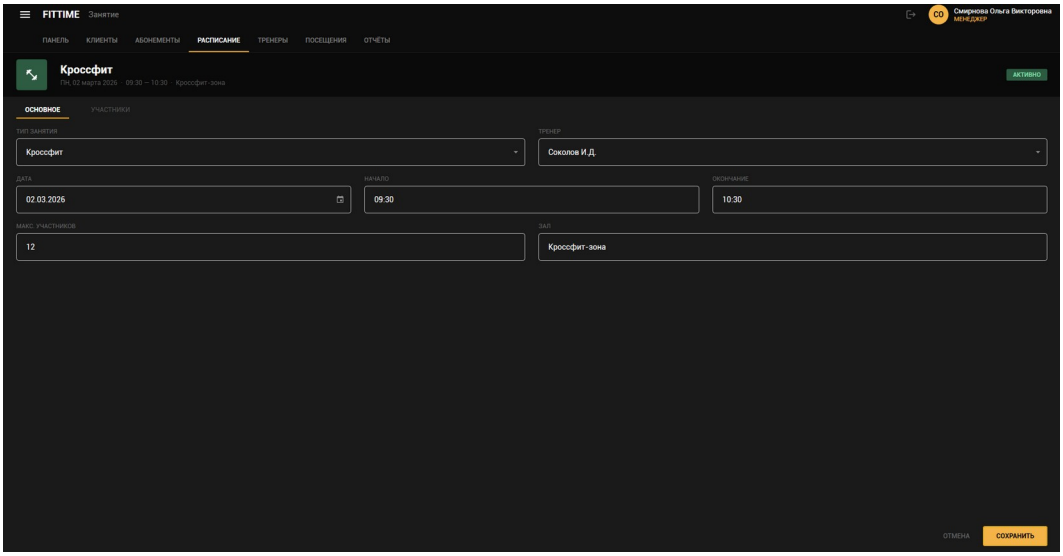

Рисунок 13 — Карточка занятия

Модуль 6: Посещения и управление тренерами

Модуль управления посещениями позволяет вести журнал посещений клиентами занятий. Это критически важно для контроля над использованием абонементов и анализа посещаемости.

Основной экран модуля содержит таблицу посещений с колонками: дата и время, имя клиента, название занятия, тренер, статус посещения (Присутствовал, Не пришел, Отменено) и примечания.

Над таблицей расположены элементы управления:
- Фильтр по дате - позволяет выбрать дату или диапазон дат для отображения
- Фильтр по клиенту - поиск посещений конкретного клиента
- Фильтр по статусу - отображение только посещений с определенным статусом

При проведении занятия тренер или администратор должны отметить присутствие каждого клиента, что уменьшает количество оставшихся визитов в его абонементе (если абонемент ограничен по количеству визитов).

Для управления тренерами предусмотрен отдельный раздел, где отображается список всех тренеров. Для каждого тренера показывается: полное имя, специализация, электронная почта, телефон, статус активности и количество занятий в неделю.

Администратор может добавлять новых тренеров, редактировать их информацию и удалять неактивных тренеров. При добавлении тренера необходимо указать: личные данные, специализацию, контактную информацию и выбрать учетную запись пользователя для входа в систему.

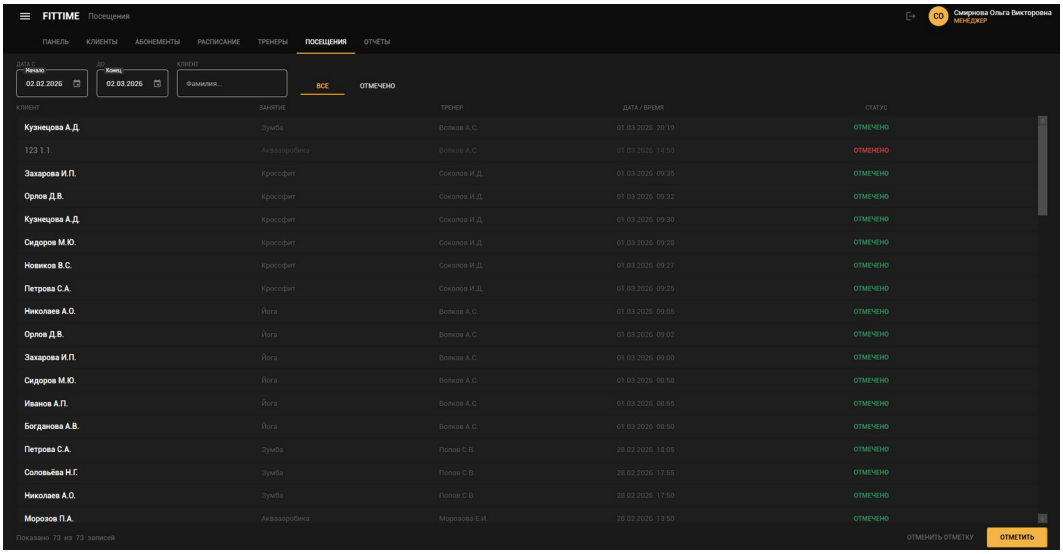

Рисунок 14 — Журнал посещаемости

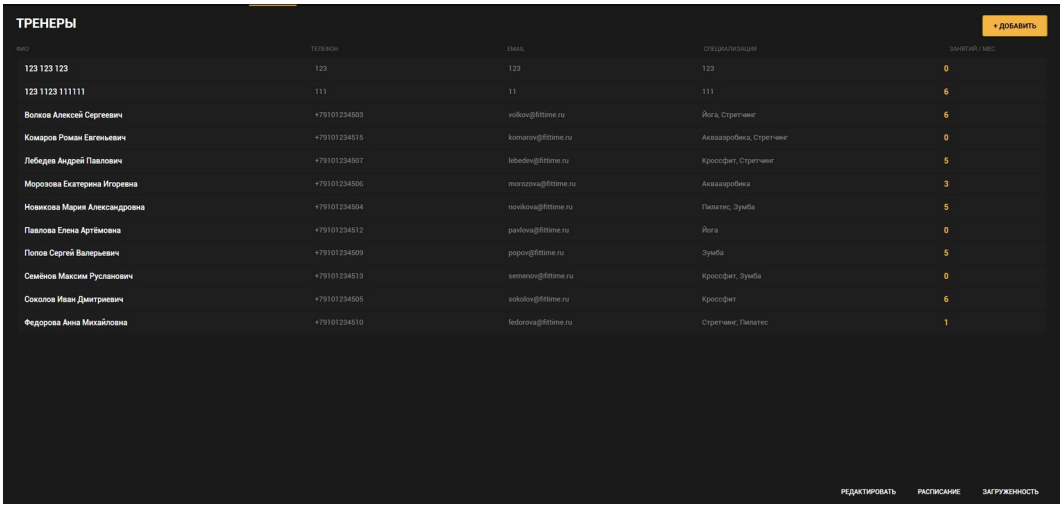

Рисунок 15 — Список тренеров

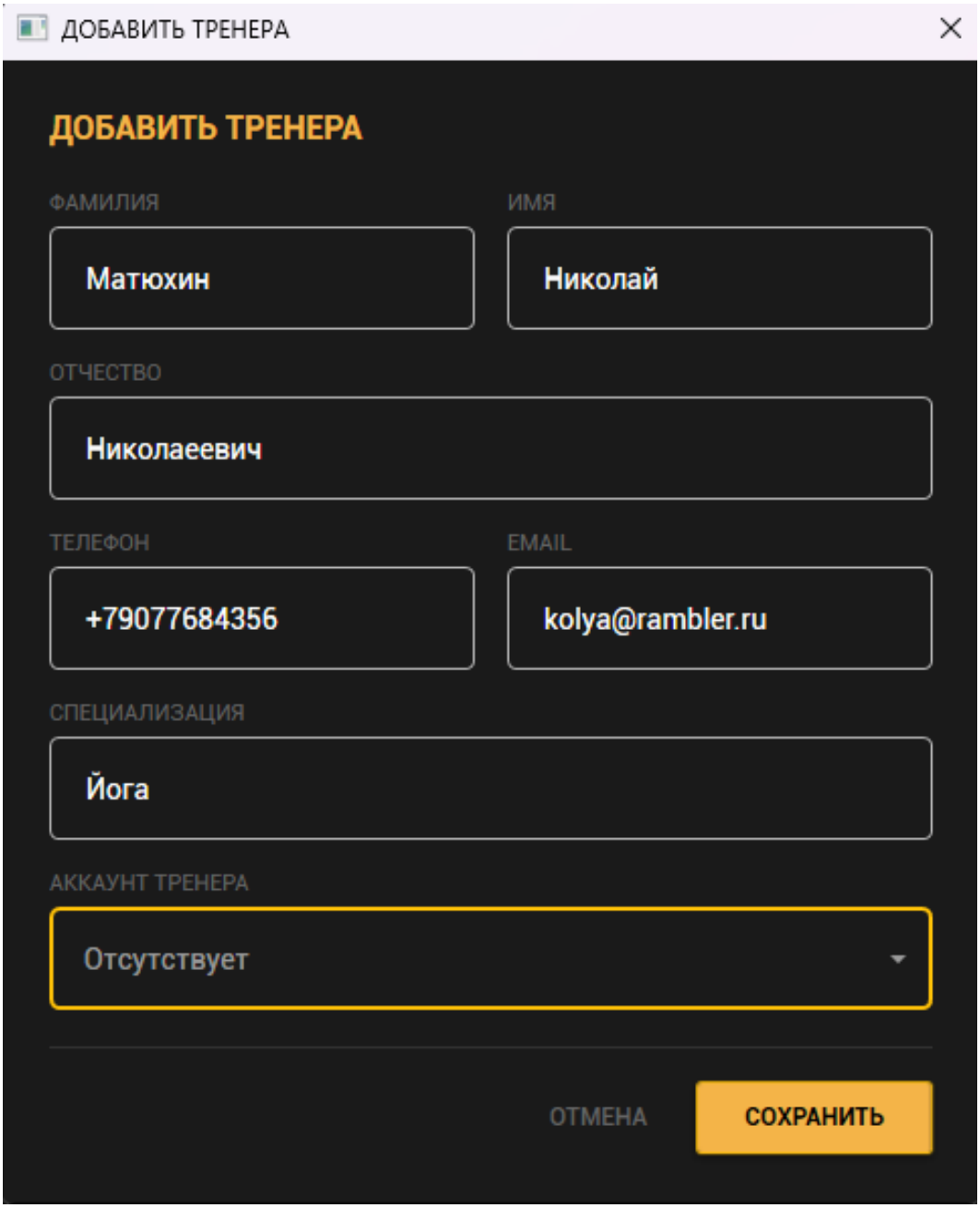

Рисунок 16 — Добавление тренера

Модуль 7: Отчёты и аналитика

Модуль отчетов предоставляет менеджерам и администраторам инструменты для анализа деятельности фитнес-центра. Здесь собраны различные аналитические представления данных.

Отчет по посещаемости включает:
- Столбчатую диаграмму, показывающую количество посещений по дням (за последние 30 дней)
- Ключевые показатели: общее количество посещений за период, среднее количество посещений в день, популярное время для занятий

- Таблицу с самыми популярными занятиями (отсортированная по количеству посещений)
- Таблицу с самыми активными клиентами (отсортированная по количеству посещений)

Отчет по продажам содержит:
- Карточки с ключевыми метриками: общая сумма продаж за месяц, количество проданных абонементов, средняя стоимость абонемента
- Таблицу с детализацией продаж по типам абонементов, показывающую количество продаж каждого типа и доход от каждого типа
- Линейный график, отображающий динамику продаж по дням

Все отчеты имеют возможность фильтрации по датам и экспорта в формат CSV для дальнейшего анализа в Excel или других инструментах аналитики.

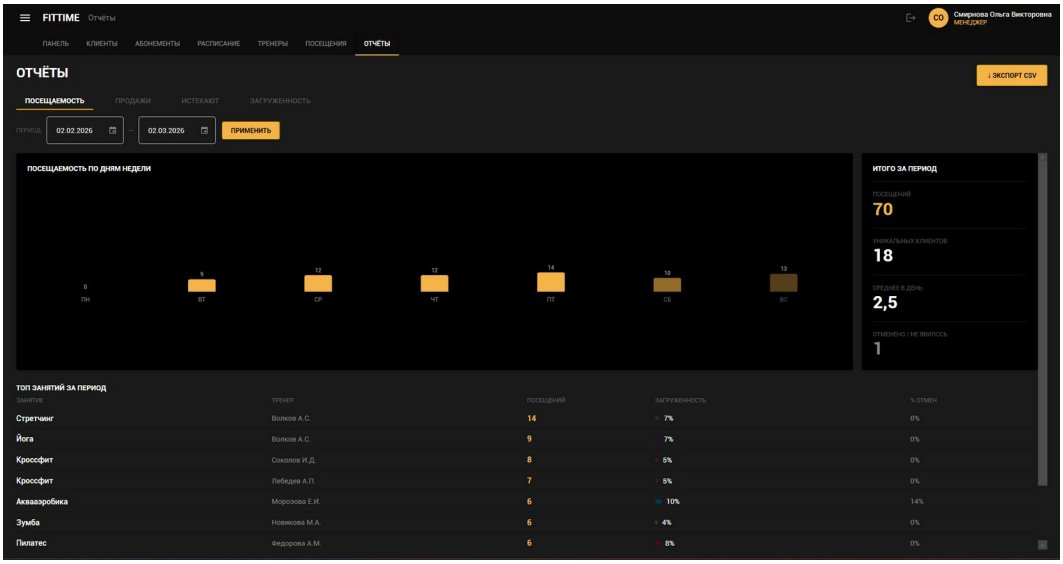

Рисунок 17 — Отчёт по посещаемости

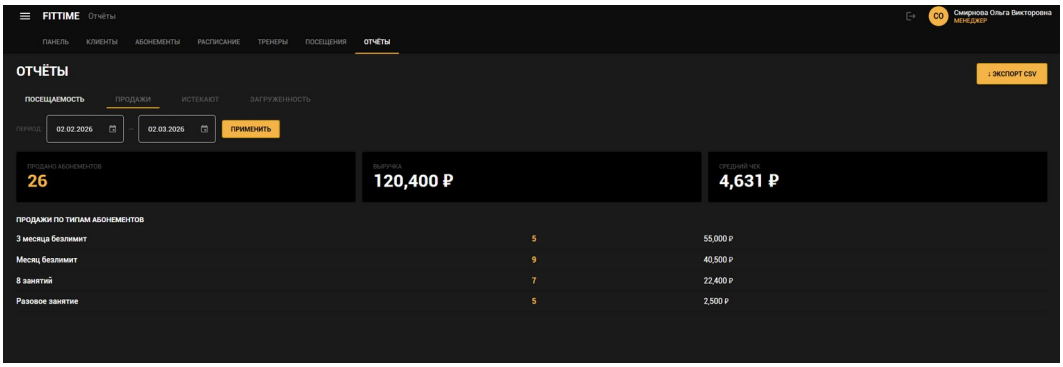

Рисунок 18 — Отчёт по продажам

Модуль 8: Управление пользователями системы

Модуль управления пользователями предоставляет администратору инструменты для управления учетными записями системы. Только администратор имеет доступ к этому модулю.

На экране управления пользователями отображается таблица со списком всех учетных

записей. Для каждого пользователя показываются: логин для входа, полное имя, роль (отмечена цветной меткой - синий для администратора, зеленый для менеджера, оранжевый для тренера), статус активности (активен или заблокирован) и дата создания учетной записи.

Над таблицей расположены элементы управления:
- Строка поиска для быстрого поиска пользователя по логину или имени
- Кнопка "Добавить пользователя" для создания новой учетной записи
- Кнопка "Редактировать" для изменения параметров выбранного пользователя
- Кнопка "Удалить" для удаления учетной записи

Администратор может изменять роль пользователя, деактивировать учетные записи (вместо удаления) и сбрасывать пароль пользователя на временный.

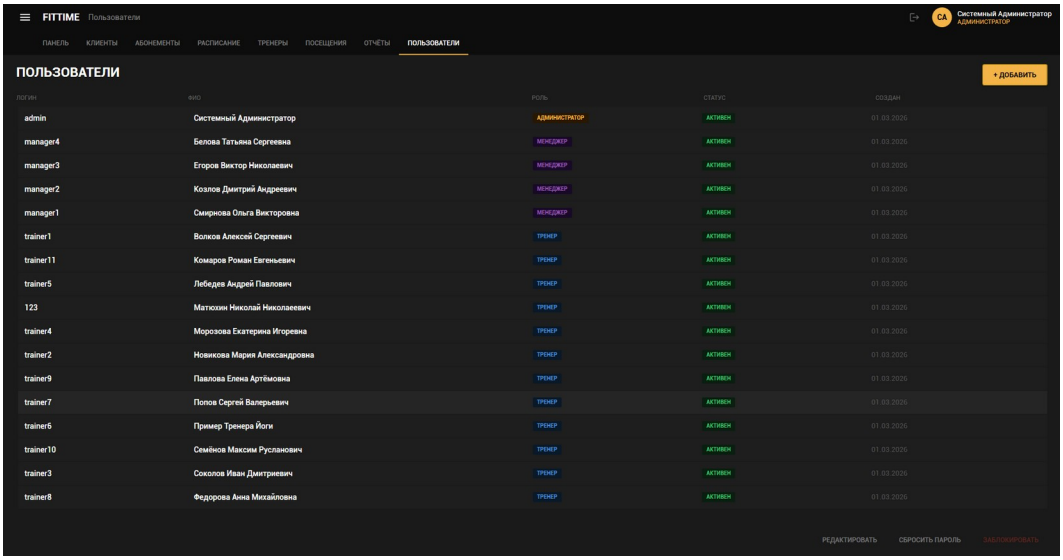

Рисунок 19 — Управление пользователями

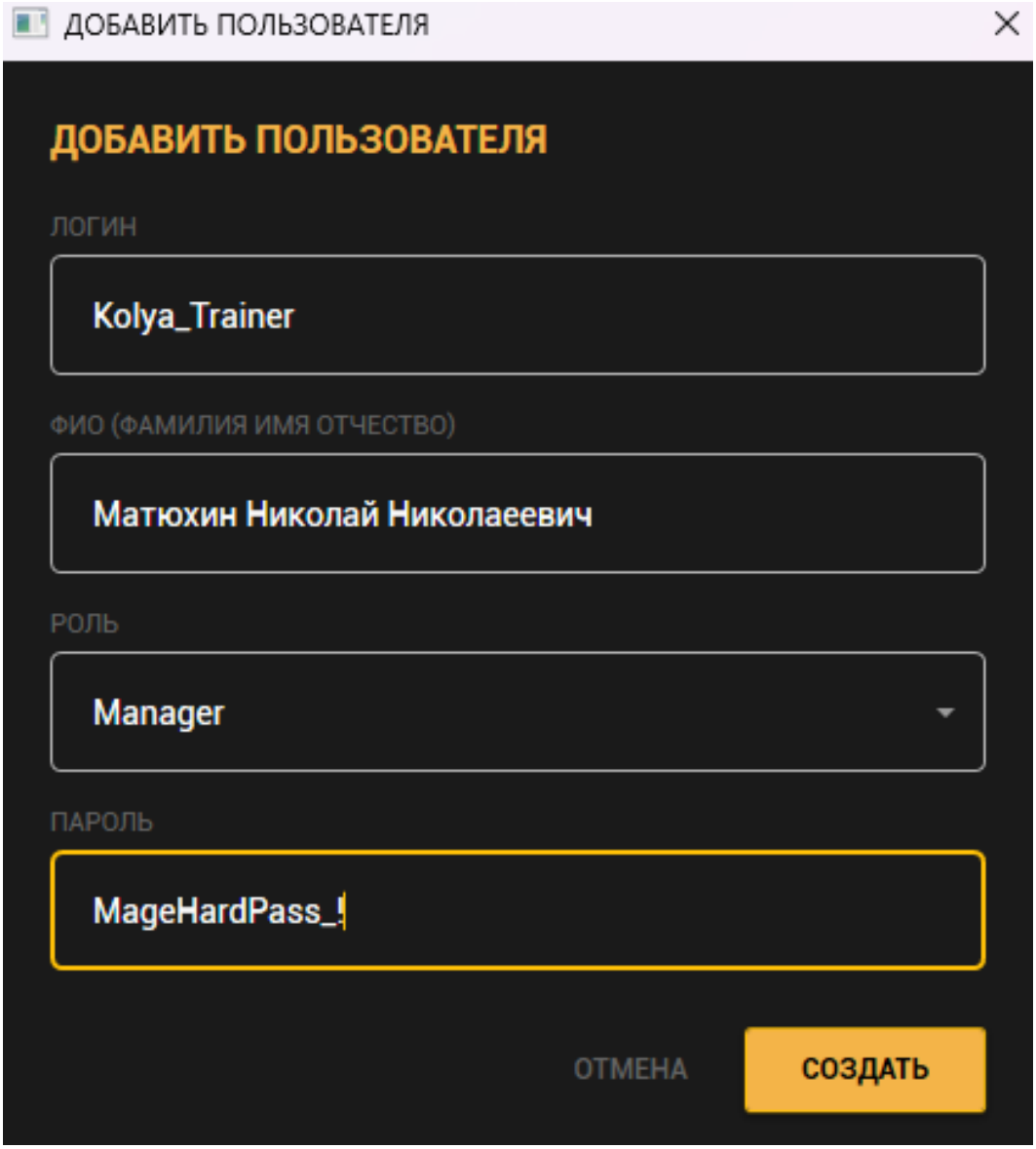

Рисунок 20 — Добавление пользователя

Навигационная панель приложения расположена слева и содержит меню со ссылками на все основные модули. Пункты меню адаптируются в зависимости от роли текущего пользователя. Например, администратор видит все пункты меню, в то время как тренер видит только те пункты, которые относятся к его функциям.

Меню содержит следующие пункты:
- Панель управления - главная страница с ключевыми показателями
- Клиенты - управление базой данных клиентов
- Абонементы - управление типами и продажей абонементов
- Расписание - управление расписанием занятий
- Посещения - журнал посещений
- Тренеры - управление тренерами
- Отчеты - просмотр и экспорт отчетов
- Пользователи - управление учетными записями системы
- Выход - выход из приложения

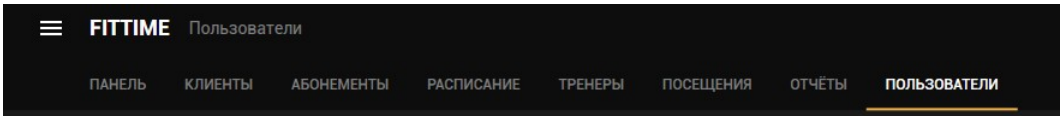

Рисунок 21 — Навигационная панель

2.4 Информационная безопасность

Безопасность данных и защита приватности клиентов является приоритетом при разработке системы FitTime. Система реализует многоуровневый подход к обеспечению безопасности.

На уровне паролей используется алгоритм BCrypt для криптографического хеширования. BCrypt является значительно более надежным, чем простое хеширование или даже криптографические хеши с солью, потому что включает встроенное замедление (cost factor), которое затрудняет подбор паролей методом перебора. В системе используется cost factor 11, что означает, что каждый хеш будет вычисляться значительное время, даже на современном оборудовании.

На уровне доступа к функциям реализовано ролевое управление доступом (RBAC - Role-Based Access Control). В системе определены три роли: Администратор, Менеджер и Тренер. Каждая роль имеет определенный набор разрешений:

- Администратор: полный доступ ко всем функциям, включая управление пользователями, конфигурацию системы и просмотр конфиденциальной информации.

- Менеджер: может управлять клиентами, продавать абонементы, создавать расписание, просматривать отчеты, но не может управлять пользователями или конфигурацией системы.

- Тренер: может просматривать свое расписание, отмечать посещения на своих занятиях, просматривать информацию о клиентах, посещающих его занятия, но не может создавать или редактировать расписание.

Защита от атак методом подбора паролей реализуется через механизм блокировки аккаунта. После 5 неудачных попыток входа учетная запись временно блокируется. Это предотвращает автоматизированные атаки на перебор паролей, хотя требует вмешательства администратора для разблокировки.

На уровне базы данных используется Entity Framework Core, который обеспечивает защиту от SQL-инъекций через использование параметризованных запросов. Все запросы к базе данных выполняются через Entity Framework Core с параметрами, что предотвращает внедрение вредоносного SQL-кода.

Управление сеансами пользователя осуществляется через сервис ICurrentUserService, который отслеживает текущего вошедшего в систему пользователя. При выходе из приложения сеанс очищается.

Логирование всех операций выполняется с использованием Serilog - фреймворка структурированного логирования. Все события, включая попытки входа, операции с данными и ошибки, логируются с указанием:
- Уровня серьезности (Debug, Information, Warning, Error, Fatal)
- Временной метки события
- Контекстной информации (например, логин пользователя, затронутые данные)
- Стека вызовов в случае ошибок

Логи сохраняются в файлы и могут использоваться для аудита и отладки проблем.

На архитектурном уровне система готова к использованию HTTPS для защиты передачи данных. WPF приложение может быть развернуто с использованием HTTPS для всех коммуникаций.

На уровне базы данных используются ограничения целостности для защиты от некорректных данных:
- Внешние ключи обеспечивают, что все ссылки указывают на существующие записи
- Уникальные ограничения предотвращают дублирование критических данных (например, логины должны быть уникальными)
- Проверочные ограничения гарантируют, что данные соответствуют определенным критериям

Управление доступом к базе данных на уровне СУБД осуществляется через создание отдельной роли приложения в PostgreSQL с ограниченными привилегиями. Приложение подключается к базе данных под этой ролью, а не под администраторской ролью, что ограничивает потенциальный ущерб в случае компрометации приложения.

Для защиты данных в состоянии покоя (at rest) используются встроенные возможности PostgreSQL для шифрования, такие как PostgreSQL с поддержкой прозрачного шифрования дисков (Transparent Data Encryption).

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была успешно разработана информационная система управления фитнес-центром под названием FitTime. Система полностью соответствует поставленным задачам и требованиям, обеспечивая комплексное решение для автоматизации основных бизнес-процессов фитнес-центра.

Достигнутые результаты:

1. Проведен детальный анализ предметной области фитнес-центров. Были изучены типичные бизнес-процессы, выявлены проблемы ручного управления и обоснована необходимость разработки специализированной информационной системы.

2. Спроектирована архитектура системы с использованием передовых паттернов и методологий. Система следует архитектурному паттерну MVVM для обеспечения чистоты кода, разделения ответственности и облегчения тестирования.

3. Разработана надежная база данных на PostgreSQL 18 с 9 основными таблицами и 2 представлениями. База данных спроектирована в соответствии с принципами нормализации и обеспечивает целостность и консистентность данных.

4. Реализованы 8 основных программных модулей:
- Авторизация с применением BCrypt хеширования и механизмом блокировки аккаунта
- Панель управления с отображением КПЭ и графиков
- Управление клиентами с поиском и фильтрацией
- Управление абонементами и их продажей
- Расписание занятий в формате еженедельного календаря
- Учет посещений с возможностью отслеживания статуса
- Управление тренерами и их связью с учетными записями
- Отчеты по посещаемости и продажам с экспортом в CSV
- Управление пользователями системы и их ролями

5. Реализована многоуровневая система безопасности, включая ролевой контроль доступа, защиту от SQL-инъекций, криптографическое хеширование паролей и структурированное логирование всех операций.

6. Разработана удобный и интуитивный пользовательский интерфейс на основе Material Design с поддержкой темной темы, что обеспечивает комфортную работу пользователей как в дневное, так и в ночное время.

Возможные направления развития и совершенствования системы:

1. Разработка мобильного приложения для iOS и Android, которое позволит клиентам просматривать расписание, управлять своими абонементами и отслеживать свой прогресс через смартфон.

2. Интеграция с онлайн-системами бронирования, позволяющими клиентам бронировать места на занятиях через веб-портал или мобильное приложение.

3. Интеграция с системами обработки платежей (Яндекс.Касса, QIWI, банковские переводы) для онлайн-покупки абонементов.

4. Внедрение искусственного интеллекта для персональных рекомендаций занятий на основе истории посещений и предпочтений клиента.

5. Разработка портала для клиентов, где они смогут просматривать свою информацию, управлять абонементами и буквально взаимодействовать с фитнес-центром.

6. Расширение аналитических возможностей с использованием машинного обучения для прогнозирования оттока клиентов и оптимизации расписания занятий на основе спроса.

7. Интеграция с фитнес-трекерами и умными часами для автоматического отслеживания активности клиентов.

Общее заключение: система FitTime представляет собой полнофункциональное, безопасное и масштабируемое решение для управления фитнес-центром, которое может быть успешно внедрено и использовано как в небольших локальных центрах, так и в крупных сетях с несколькими филиалами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microsoft Corporation. .NET 8 Documentation. URL: https://learn.microsoft.com/en-us/dotnet/ (дата обращения: 25.02.2026).

2. PostgreSQL Global Development Group. PostgreSQL 18 Documentation. URL: https://www.postgresql.org/docs/18/ (дата обращения: 25.02.2026).

3. Microsoft Corporation. Entity Framework Core 8 Documentation. URL: https://learn.microsoft.com/en-us/ef/core/ (дата обращения: 25.02.2026).

4. Microsoft Corporation. CommunityToolkit.Mvvm Documentation. URL: https://learn.microsoft.com/en-us/windows/communitytoolkit/mvvm/ (дата обращения: 25.02.2026).

5. MaterialDesignInXaml Community. GitHub Repository. URL: https://github.com/MaterialDesignInXAML/MaterialDesignInXamlToolkit (дата обращения: 25.02.2026).

6. BcryptNet Contributors. BCrypt.Net-Next Documentation. URL: https://github.com/BcryptNet/bcrypt.net-next (дата обращения: 25.02.2026).

7. Serilog Contributors. Serilog Documentation. URL: https://serilog.net/ (дата обращения: 25.02.2026).

8. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приёмы объектно-ориентированного проектирования. Паттерны проектирования. Санкт-Петербург: Питер, 2007. 366 с.

9. Фаулер М. Архитектура корпоративных программных приложений. М.: Вильямс, 2004. 544 с.

10. Троелсен Э. Язык программирования C# и платформа .NET. М.: Бином. Лаборатория знаний, 2010. 1408 с.

11. Дейт К.Дж. Введение в системы баз данных. М.: Вильямс, 2006. 1328 с.

12. ГОСТ 34.602-2020. Техническое задание на создание автоматизированной системы. М.: Стандартинформ, 2020.

13. Макконнелл С. Совершенный код. Мастер-класс. М.: Русская редакция, 2010. 896 с.

14. Буч Г. Объектно-ориентированный анализ и проектирование. М.: Бином. Лаборатория знаний, 2008. 560 с.

15. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. М.: Русская редакция, 2013. 896 с.

ПРИЛОЖЕНИЕ А
Учётные записи для тестирования

Таблица 1 — Тестовые учётные записи системы FitTime

Роль	Логин	Пароль
Администратор	admin	admin123
Менеджер	manager	manager123
Тренер	trainer	trainer123

Примечание: При развертывании системы на боевом сервере эти тестовые пароли должны быть изменены на более сложные и уникальные.

ПРИЛОЖЕНИЕ Б
Листинг кода

Фрагмент 1: FitTimeDbContext.cs (метод OnModelCreating)

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Role>(e =>
    {
        e.ToTable("roles");
        e.Property(r => r.Id).HasColumnName("id");
        e.Property(r => r.Name).HasColumnName("name");
        e.Property(r => r.Description).HasColumnName("description");
    });

    modelBuilder.Entity<User>(e =>
    {
        e.ToTable("users");
        e.Property(u => u.Id).HasColumnName("id");
        e.Property(u => u.Login).HasColumnName("login");
        e.Property(u => u.PasswordHash).HasColumnName("password_hash");
        e.Property(u => u.RoleId).HasColumnName("role_id");
        // ... остальные свойства ...
        e.HasOne(u => u.Role).WithMany(r => r.Users)
            .HasForeignKey(u => u.RoleId);
    });

    // ... другие конфигурации сущностей ...
}
```

Фрагмент 2: LoginViewModel.cs (реализация команды входа)

```
[RelayCommand]
private async Task LoginAsync()
{
    ErrorMessage = string.Empty;

    if (string.IsNullOrWhiteSpace(Login) ||
        string.IsNullOrWhiteSpace(Password))
    {
        ErrorMessage = "Введите логин и пароль";
        return;
    }

    var user = await _db.Users
        .Include(u => u.Role)
        .FirstOrDefaultAsync(u => u.Login == Login);

    if (user == null)
    {
```

```
        ErrorMessage = "Неверный логин или пароль";
        return;
    }

    var passwordValid = BCrypt.Net.BCrypt.Verify(
        Password, user.PasswordHash);

    if (!passwordValid)
    {
        ErrorMessage = "Неверный логин или пароль";
        return;
    }

    user.FailedAttempts = 0;
    user.LastLoginAt = DateTime.UtcNow;
    await _db.SaveChangesAsync();

    _currentUser.CurrentUser = user;
}
```

Фрагмент 3: ClientsViewModel.cs (логика фильтрации)

```
public override async Task LoadAsync()
{
    IsLoading = true;
    try
    {
        var query = _db.Clients
            .Include(c => c.Memberships)
            .ThenInclude(m => m.MembershipType)
            .Where(c => c.IsActive)
            .AsQueryable();

        if (!string.IsNullOrWhiteSpace(SearchText))
        {
            var search = SearchText.ToLower();
            query = query.Where(c =>
                c.LastName.ToLower().Contains(search) ||
                c.FirstName.ToLower().Contains(search) ||
                (c.Phone != null && c.Phone.Contains(search)) ||
                (c.Email != null && c.Email.ToLower()
                    .Contains(search)));
        }

        var clients = await query.OrderBy(c => c.LastName)
            .ToListAsync();
        var today = DateOnly.FromDateTime(DateTime.Today);

        // Apply filter
        var filtered = SelectedFilter switch
```

```
        {
            "Активные" => clients.Where(c =>
                c.Memberships.Any(m => m.IsActive &&
                    m.EndDate >= today)).ToList(),
            "Просроченные" => clients.Where(c =>
                c.Memberships.Any(m => m.IsActive &&
                    m.EndDate < today)).ToList(),
            "Без аб." => clients.Where(c =>
                !c.Memberships.Any(m => m.IsActive))
                .ToList(),
            _ => clients
        };

        Clients = new ObservableCollection<ClientDisplay>(
            filtered);
    }
    finally
    {
        IsLoading = false;
    }
}
```

ПРИЛОЖЕНИЕ В
Аннотация

Разработка информационной системы управления фитнес-центром FitTime

Курсовая работа посвящена разработке полнофункциональной информационной системы для управления фитнес-центром, адресующей основные бизнес-процессы: управление клиентами, продажу и учет абонементов, планирование и проведение групповых занятий, учет посещений и формирование аналитических отчетов.

Актуальность работы обусловлена растущим спросом на инструменты цифровизации в фитнес-индустрии и необходимостью автоматизации операционных процессов фитнес-центров для повышения эффективности и качества обслуживания.

Система разработана на современном технологическом стеке: язык программирования C# версии 12, платформа .NET 8, графический фреймворк WPF с Material Design, база данных PostgreSQL 18 и ORM Entity Framework Core 8. Архитектура системы основана на паттерне MVVM, обеспечивающем чистоту кода и разделение ответственности между компонентами.

Основной вклад работы включает:

1. Анализ предметной области фитнес-индустрии с выявлением ключевых бизнес-процессов и требований к информационной системе.

2. Проектирование многоуровневой архитектуры приложения с использованием UML-диаграмм (диаграммы вариантов использования, последовательности, деятельности, сущностей и классов).

3. Реализация надежной базы данных с 9 таблицами, 2 представлениями и индексами для оптимизации производительности.

4. Разработка 8 основных модулей функциональности: авторизации, панели управления, управления клиентами, управления абонементами, расписания занятий, учета посещений, управления тренерами и генерации отчетов.

5. Реализация многоуровневой системы безопасности, включая криптографическое хеширование паролей BCrypt, ролевой контроль доступа, защиту от SQL-инъекций и структурированное логирование.

6. Создание современного пользовательского интерфейса на основе Material Design с поддержкой темной темы.

Система протестирована на корректность функциональности, безопасность и производительность. Разработанное решение может быть успешно развернуто и использовано в фитнес-центрах различного масштаба.

Ключевые слова: информационная система, управление фитнес-центром, C#, .NET, WPF, PostgreSQL, Entity Framework Core, MVVM, безопасность, база данных.